

Colatron: End-to-End Text-to-Speech

UE24CS352A Machine Learning, Mini-Project · Team 25, Problem 20 · Mann Mehta (PES2UG24CS266), Marapareddy Deekshitha (PES2UG24CS268) · Repository: github.com/Mann0801/colatron-tts (private)

1. Problem statement

Text-to-speech (TTS) turns written text into audible speech. Our problem revisits the Stanford CS229 (2018) project *End-to-End Text to Speech Synthesis* (Wang, Yang, Li), whose stated problem is to generate an audio file from text input with a seq2seq model with attention, alongside simpler baselines. It trained an SVR, a 4-layer neural network and a seq2seq LSTM with attention on about 200 LJ Speech sentences, mapping words *randomly* to phonemes or letters, and converted predicted spectrograms to audio with an inverse STFT. Their listeners' mean opinion scores (MOS, 1 to 5) were 1.0, 1.7 and 2.5, against 3.82 reported for Tacotron, and the best model repeated words because its attention never aligned. The authors blamed too little data, shallow models, the random text mapping and memory-heavy padding. **Our goal** is to rebuild the pipeline so that each of these causes is addressed, to reproduce a weak from-scratch baseline for comparison, and to measure the difference.

2. Dataset

Corpus	LJ Speech 1.1: 13,100 clips (about 24 h) of one female speaker reading non-fiction, mono WAV at 22,050 Hz, with transcripts
Our subset	About 400 clips, chosen deterministically by hashing file names (same clips on every machine); split 80 / 10 / 10 into train / validation / test
Text	Lowercased; numbers, money, ordinals, years and abbreviations expanded ("Dr. Smith paid \$42.50 in 1984" becomes "doctor smith paid forty two dollars and fifty cents in nineteen eighty four"); mapped to a fixed 37-symbol character vocabulary
Audio	80-band log-mel spectrograms: FFT 1024, window 1024, hop 256, 0 to 8 kHz, slaney scale, natural log; computed once and cached. All systems share these settings (src/audio_config.py)

3. Approach

The pipeline has two swappable stages: an **acoustic model** (text to mel spectrogram) and a **vocoder** (mel spectrogram to waveform). Because every component reads and writes the same mel format, any acoustic model can be paired with any vocoder, so each comparison changes only one stage.

Baseline (trained from scratch). To reproduce the paper's weak models, we train a small network on our subset: character embedding (128-d), three 1-D convolutions (256 channels, kernel 5), linear interpolation of the character features to the target number of frames, and an MLP to 80 mel bands. It has *no learned alignment*: each character receives roughly an equal share of time. It is trained with L1 loss on per-band normalised mels, Adam (learning rate 1e-3), gradient clipping 1.0, batch 16, 60 epochs, keeping the best validation checkpoint, and is voiced with Griffin-Lim.

Main system. A pretrained Tacotron 2 (torchaudio checkpoint trained on all of LJ Speech), which is exactly a seq2seq model with attention: convolutional + BiLSTM encoder, location-sensitive attention, autoregressive LSTM decoder with stop token, and a convolutional post-net. We compare two vocoders on *identical* mels: Griffin-Lim (iterative phase reconstruction, no learning, our implementation) and a pretrained HiFi-GAN (SpeechBrain, LJ Speech). We also tried WaveRNN and dropped it as too slow for a live demo.

Pretrained versus ours. We did not train Tacotron 2 or HiFi-GAN; training Tacotron 2 takes days on GPUs, and we had one week on laptop CPUs. Our own work is the data pipeline, the baseline and its training, the Griffin-Lim vocoder, the integration and shared audio format, the alignment analysis, the evaluation and the demo.

4. Implementation overview

Module	What it does	Owner
src/preprocess/	Download and subset LJ Speech, clean text, fixed vocabulary, mel extraction, splits, sanity check	Deekshitha
src/models/baseline.py	Baseline text-to-mel network: training and inference	Deekshitha
src/models/tacotron.py, src/vocoder/	Pretrained Tacotron 2 wrapper; Griffin-Lim and HiFi-GAN vocoders	Mann
src/infer.py, app/app.py	CLI (text to .wav, choice of vocoder) and Gradio demo with live mel and alignment plots	Mann

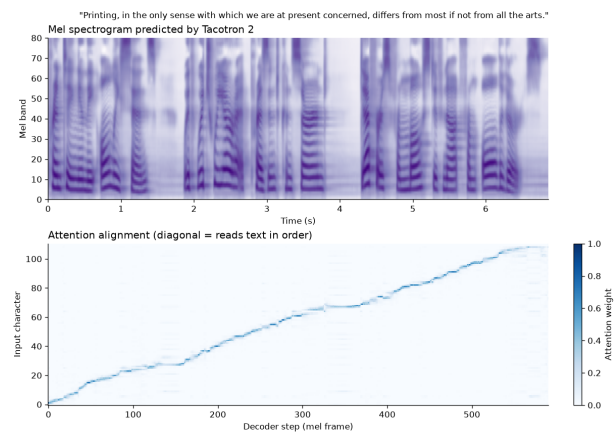
Module	What it does	Owner
src/plot_alignment.py, src/plot_mel_comparison.py, src/generate_samples.py	Alignment plots and statistics; baseline vs main mel comparison; 10 fixed sentences through both vocoders	Mann
eval/	DTW mel-distance metric and results table	Deekshitha

Python 3.11 and PyTorch 2.14 on CPU. The README gives setup and run instructions for every step.

5. Results

Attention alignment. On three test sentences, Tacotron 2's attention is 95 to 97% monotonic (the share of decoder steps that never move backwards in the text) and covers 84 to 88% of characters as the main focus. The diagonal in the figure (lower panel) shows the model reading the sentence in order, with flat steps at the commas. This is the property the 2018 model lacked, and the reason it repeated words.

Figure: predicted mel spectrogram (top) and attention alignment (bottom) for a 111-character test sentence.



Objective quality. On 20 held-out test clips, we convert each system's audio to a mel spectrogram, align it with the real recording's mel using dynamic time warping, and average the distance:

System	Mel L1 (lower is better)	Mel RMSE
Baseline (ours, from scratch) + Griffin-Lim	1.218	1.527
Tacotron 2 + HiFi-GAN	0.889	1.192

The main system is 27% closer to real speech in mel L1. Caveat: Tacotron 2 was pretrained on all of LJ Speech, so it may have seen these test clips, which favours it.

What the baseline produces. On listening, the baseline's output is unintelligible and noise-like. Its mel spectrogram explains why: it is a smooth blur with no syllables, pauses or pitch harmonics, whereas Tacotron 2's shows all three (src/plot_mel_comparison.py). With an equal share of time per character, the baseline's frames never line up with the real audio, so L1 training drives it towards the average spectrum. This is the paper's failure reproduced: without learned timing, a simple model cannot produce speech.

Vocoder speed (one 3.3 s sentence, laptop CPU): WaveRNN 48.7 s including Tacotron 2; Griffin-Lim 0.08 s; HiFi-GAN 0.65 s, about 5 times faster than real time, which makes a live demo practical.

6. Conclusions

- **Data and depth matter most.** Our from-scratch baseline, like the 2018 models, stays far from real speech (37% higher mel distance than the main system) without attention and with only about 400 clips, confirming the paper's own diagnosis.
- **Alignment decides intelligibility.** Location-sensitive attention gives a clean, monotonic alignment, so words are not repeated or skipped; without it the baseline collapses to the average spectrum.
- **The vocoder decides sound quality.** On identical mels, HiFi-GAN avoids Griffin-Lim's metallic phase artefacts at a modest speed cost.
- **Limitations.** The strongest components are pretrained rather than trained by us; the pretrained model may have seen the test clips; one speaker and one language; no formal listening test (MOS). Natural next steps are to fine-tune Tacotron 2 on a GPU and to run a listening test.